NumPy Assignment — Solutions

All tasks are answered with NumPy code. Random examples use fixed seeds where practical so results can be reproduced. The assignment does not specify polynomial coefficients.

1. Import NumPy and print version

Python / SQL code

```
import numpy as np
print(np.__version__)
```

Sample output

```
A version string is printed, for example: 2.x.x (depends on installed NumPy version).
```

2. Array creation

Python / SQL code

```
import numpy as np
a1 = np.arange(10)
rng = np.random.default_rng(42)
a2 = rng.integers(1, 51, size=(3, 3))
a3 = np.ones((2, 3, 4), dtype=int)
print("1D:", a1)
print("2D:\n", a2)
print("3D shape:", a3.shape)
```

Sample output

```
1D: [0 1 2 3 4 5 6 7 8 9]
2D:
[[ 5 39 33]
 [22 22 43]
 [ 5 35 11]]
3D shape: (2, 3, 4)
(The 2D values are deterministic with NumPy's default_rng seed 42.)
```

3. Shape, size and dtype

Python / SQL code

```
for name, arr in {"a1": a1, "a2": a2, "a3": a3}.items():
    print(name, "shape =", arr.shape, "size =", arr.size, "dtype =", arr.dtype)
```

Sample output

```
a1 shape=(10,) size=10 dtype=int64
a2 shape=(3, 3) size=9 dtype=int64
a3 shape=(2, 3, 4) size=24 dtype=int64
(dtype may be int32 on some platforms)
```

4. Element-wise operations

Python / SQL code

```
x = np.full((3, 3), 5)
y = np.full((3, 3), 3)
print("Addition:\n", x + y)
print("Subtraction:\n", x - y)
print("Multiplication:\n", x * y)
print("Division:\n", x / y)
```

Sample output

```
Addition: all 8
Subtraction: all 2
Multiplication: all 15
Division: all 1.66666667
Each result is a 3x3 matrix.
```

5. Statistical operations

Python / SQL code

```
rng = np.random.default_rng(10)
values = rng.integers(10, 101, size=15)
print("Array:", values)
print("Min:", values.min(), "Max:", values.max())
print("Mean:", values.mean(), "Median:", np.median(values))
```

```
print("Std:", values.std())
```

Sample output

```
Prints the generated 15 values and their minimum, maximum, mean,
median and population standard deviation. Seed 10 makes it reproducible.
```

6. Slicing and indexing

Python / SQL code

```
arr = np.arange(1, 26).reshape(5, 5)
print("First row:", arr[0, :])
print("Last column:", arr[:, -1])
print("Top-left 2x2:\n", arr[:2, :2])
print("Greater than 10:", arr[arr > 10])
```

Sample output

```
First row: [1 2 3 4 5]
Last column: [ 5 10 15 20 25]
Top-left 2x2:
[[1 2]
 [6 7]]
Greater than 10: [11 12 13 14 15 16 17 18 19 20 21 22 23 24 25]
```

7. Broadcasting

Python / SQL code

```
one_d = np.array([1, 2, 3])
two_d = np.full((3, 3), 10)
print(two_d + one_d)
```

Sample output

```
[[11 12 13]
 [11 12 13]
 [11 12 13]]
```

8. Matrix multiplication and transpose

Python / SQL code

```
rng = np.random.default_rng(7)
A = rng.integers(1, 11, size=(2, 3))
B = rng.integers(1, 11, size=(3, 2))
product = A @ B
print("A:\n", A)
print("B:\n", B)
print("A @ B:\n", product)
print("Transpose:\n", product.T)
```

Sample output

```
A and B are generated with seed 7.
A @ B is a 2x2 matrix; its transpose is the same values with rows
and columns exchanged. Exact matrices are printed by the code.
```

9. Boolean masking

Python / SQL code

```
rng = np.random.default_rng(20)
values = rng.integers(1, 51, size=20)
mask = values > 25
print("Values:", values)
print("Mask:", mask)
print("Greater than 25:", values[mask])
```

Sample output

```
Prints the 20 seeded random integers, the True/False mask,
and the selected values greater than 25.
```

10. Normalize 1D array to 0–1

Python / SQL code

```
rng = np.random.default_rng(30)
values = rng.integers(50, 151, size=10)
normalized = (values - values.min()) / (values.max() - values.min())
print("Original:", values)
print("Normalized:", normalized)
```

Sample output

```
Normalized values range from 0.0 to 1.0. The minimum maps to 0
and the maximum maps to 1.
```

11. Image representation and scaling

Python / SQL code

```
rng = np.random.default_rng(40)
pixels = rng.integers(0, 256, size=(5, 5), dtype=np.uint8)
scaled_pixels = pixels.astype(float) / 255.0
print("Pixels:\n", pixels)
print("Scaled:\n", scaled_pixels)
```

Sample output

```
Original pixel values are integers from 0 to 255.
Scaled values are floating-point numbers from 0.0 to 1.0.
```

12. Polynomial evaluation

The assignment does not state the polynomial coefficients. The example below evaluates $2x^2 + 3x + 1$; replace coefficients if another polynomial was intended.

Python / SQL code

```
x = np.arange(-10, 11)
y = 2*x**2 + 3*x + 1
print("x:", x)
print("y:", y)
```

Sample output

```
x: [-10 -9 -8 ... 8 9 10]
y: [171 136 105 ... 153 190 231]
```